

Raport: Generator serii questów PDDL

Autorzy:

- Paulina Hładki (151781)
- Marta Jędrzejczak (151773)
- Szymon Szymankiewicz (151821)
- Dominik Maćkowiak (151915)

1. Artefakty

Projekt zawiera cztery wymagane elementy:

- Kod generatora fabuły: `questgen/generator.py`, wspierany przez `questgen/domain.py` i `questgen/pddl.py`.
- Folder z definicjami questów: `quests/generated/ash_bell/`.
- Kod odgrywający fabuły: `questgen/play.py`.
- Raport podsumowujący: ten dokument.

Wygenerowany folder zawiera:

- `domain.pddl` — wspólna domena świata.
- `prompt.txt` — oryginalny angielski prompt.
- `series.json` — pełna wygenerowana seria.
- `quest_01_quest-1.json`, `quest_02_quest-2.json`, `quest_03_quest-3.json` — opisy questów, obiekty, NPC i dialogi.
- `quest_01_quest-1.pddl`, `quest_02_quest-2.pddl`, `quest_03_quest-3.pddl` — problemy planowania.
- `quest_*.plan` — plany zweryfikowane przez lokalny solver STRIPS.
- `generation_meta.json` — metadane generacji.

2. Metoda generacji

Metoda jest hybrydowa. Lokalny model językowy tworzy zwarty *dramatic seed*, podczas gdy deterministyczny kod symboliczny kompiluje ten seed do JSON i PDDL. Dzięki temu nie prosimy modelu o pisanie surowego PDDL, co jest zbyt kruche. Model językowy dostarcza tytuł, premise, motywy, tytuły questów, streszczenia i twist. Generator buduje następnie zweryfikowaną strukturę questów z lokacjami, przedmiotami, NPC, wrogami i krokami.

Pierwsza próba użyła modelu `qwen3.6:latest`, ale okazał się zbyt ciężki dla laptopa w tej konfiguracji. Wielokrotnie kończył się timeoutem i dodatkowo produkował długie myślenie przed zwróceniem użytecznego JSON. Finalne uruchomienie używa znacznie mniejszego modelu `gemma3:4b`. Ukończyło się ono pomyślnie:

```
{
  "model": "gemma3:4b",
  "used_fallback": false,
  "generation_strategy": "ollama_compact_draft_then_symbolic_pddl_compilation"
}
```

Generator wspiera trzy tryby:

- `compact` — domyślny; prosi LLM o krótki *dramatic seed*, a następnie kompiluje go symbolicznie.
- `full` — eksperymentalny; prosi LLM o pełny schemat questu.
- `fallback` — deterministyczny angielski fallback używany, gdy model jest niedostępny.

3. Wspólna domena PDDL

Wspólna domena używa typów:

- `location`
- `item`
- `npc`
- `enemy`
- `gate`
- `stage`

Akcje STRIPS reprezentują działania dostępne w grze tekstowej:

- `travel`
- `take`
- `talk`
- `give`
- `unlock`
- `fight`
- `ritual`
- `examine-location`
- `examine-item-at`
- `examine-carried-item`
- `examine-npc`
- `examine-enemy`
- `reveal-alternate-path`
- `travel-alternate`
- `take-optional`
- `press-npc`
- `brawl`
- `use-healing-item`

Każda akcja ma *preconditions* i *effects*. Zamierzona kolejność narracyjna jest reprezentowana przez predykaty stage takie jak `current-stage`, `next-stage`, `travel-step`, `take-step`, `fight-step` i `ritual-step`. Dzięki temu quest pozostaje problemem planowania, a solver nie może pominąć zamierzonej kolejności fabularnej. Akcje `examine-*` dodają opcjonalne wybory inspekcji. Akcje konsekwencji dodają decyzje ze zmianami stanu: ukryte ścieżki mogą zostać odkryte, opcjonalne przedmioty mogą zostać podniesione, NPC mogą zostać sprowokowani, opcjonalni wrogowie mogą stać się wrogo nastawieni, walki na pięści mogą zranić gracza, a przedmioty lecznicze mogą usunąć ranę.

4. Naprawa i weryfikacja

Generator nie ufa bezpośrednio wyjściu modelu. Normalizuje ID, filtruje nieobsługiwane akcje, dodaje brakujące obiekty i kompiluje kroki questów do początkowych faktów PDDL.

Repair pass może:

- dodać brakujące lokacje, NPC, przedmioty, wrogów i bramy,
- wnioskować `start_location`,
- dodać fakty `path` dla ruchu,
- dodać fakty `locked-gate` dla zamkniętych przejść,
- dodać fakty `hidden-route`, `optional-item`, `healing-item`, `provocation-open` i `optional-enemy` dla konsekwencji decyzji,
- wnioskować niesione przedmioty przez `has(item)` gdy przedmiot przechodzi z poprzedniego questu,
- dodać domyślne dialogi dla NPC,
- porównać plan solvera z zamierzonym planem narracyjnym.

Finalna weryfikacja:

```
OK    quest_01_quest-1.pddl: 5 akcji
OK    quest_02_quest-2.pddl: 7 akcji
OK    quest_03_quest-3.pddl: 6 akcji
```

5. Wygenerowana seria

Oryginalny prompt:

After years away, the hero returns to their childhood village and discovers that an ancient bell beneath the mine is waking ash-born shadows. The story should feel like a dark fairy tale, but end with hope for the village.

Wygenerowany tytuł serii:

Echoes of the Ashwood

Wygenerowany przez LLM *dramatic seed*:

- Motifs: `loss`, `memory`, `corruption`
- Twist: cienie nie są złośliwe, lecz echem zapomnianego paktu zawartego dla ochrony wioski przed znacznie większym zagrożeniem.

Sekwencja questów:

1. **The Silent Toll** — wezwanie do przygody; bohater rozmawia z Elder Mirą, znajduje zardzewiały medalion i bierze srebrny nóż.
2. **Beneath Blackstone** — próg; bohater daje medalion Iren Kartografce, zdobywa mapę żył i klucz, i otwiera bramę kopalni.
3. **The Bloom of Remembrance** — próba i powrót; bohater pokonuje strażnika popiołu, słucha upiora dzwonnika, zdobywa serce dzwonu i wykonuje rytuał ciszy.

6. Eksperyment Multi-Prompt

Aby przetestować odporność generatora, trzy różne angielskie prompty zostały przekazane modelowi `gemma3:4b`. Dwa uruchomienia użyły domyślnego trybu `compact`, a jedno eksperymentalnego trybu `full`. Wszystkie wyniki zostały zweryfikowane przez lokalny solver STRIPS, a następnie automatycznie przeszło przez text player.

Motyw promptu	Tryb	Wygenerowany tytuł serii	LLM się udało?	Podstawowa struktura questów
Zhańbiony kapitan morski + przeklęty sekstant	compact	<i>The Salt-Kissed Curse</i>	Tak (used_fallback: false)	Identyczna z szablonem fallback
Astronauta na księżycu Jowisza + obcy sygnał	full	<i>Echo of the Ashen Bell</i>	Nie (used_fallback: true)	Szablon fallback (LLM wyprodukował nierozwiązywalny schemat)
Muzyk dziedziczny nawiedzony teatr operowy	compact	<i>Echoes of Porcelain</i>	Tak (used_fallback: false)	Identyczna z szablonem fallback

Obserwacja 1 — Skóra narracyjna się zmienia, struktura nie. W każdym udanym uruchomieniu compact model gemma3:4b wyprodukował nowy *dramatic seed* (tytuł, premise, motywy, twist, tytuły questów), ale kompilator symboliczny następnie zbudował faktyczne kroki questów z tego samego deterministycznego szablonu fallback. W konsekwencji lokacje, przedmioty, NPC, wrogowie i kolejność kroków pozostały identyczne we wszystkich trzech promptach. Model przyozdobił szkielet innymi słowami, ale nie zaprojektował nowego szkieletu.

Obserwacja 2 — Tryb full jest niepraktyczny dla tego modelu. Gdy poproszono o kompletny schemat questu w trybie full, gemma3:4b wyprodukował strukturę, która nie przeszła testu rozwiązywalności plannera nawet po *repair pass*. Generator automatycznie odrzucił ten szkic i przełączył się na deterministyczny szablon. Potwierdza to, że proszenie modelu o 4 miliardy parametrów o pisanie poprawnych, rozwiązywalnych grafów questów kompatybilnych z PDDL jest obecnie niepewne.

Obserwacja 3 — Wszystkie wygenerowane serie są grywalne. Niezależnie od tego, czy LLM przyczynił się do generacji czy nie, każdy quest w każdej serii przeszedł solver STRIPS, a *playthrough --auto* ukończyło każdą serię bez błędów.

7. Odtwarzanie

Text player wczytuje zarówno JSON, jak i PDDL. JSON dostarcza opisów, nazw, narracji i dialogów. Stan PDDL decyduje, które akcje są aktualnie dostępne. Menu jest generowane z możliwych do wykonania *grounded* akcji STRIPS, a nie z ręcznie napisanego skryptu.

Pierwsza implementacja była zbyt liniowa: każdy stan zazwyczaj oferował tylko jedną akcję. Obecna wersja naprawia to za pomocą opcjonalnych akcji STRIPS i akcji konsekwencji. Na przykład pierwszy stan teraz oferuje:

Available actions:

1. Talk to: Elder Mira
2. Examine: Heatherfall
3. Observe: Elder Mira

Późniejsze wybory mogą zmieniać stan gry. Przykłady:

Available actions:

1. Go to: Ancestor Shrine
2. Examine: Old Well
3. Examine carried item: Rusted Medallion
4. Reveal hidden path with: Rusted Medallion

W drugim queście naciśnięcie Iren tworzy wrogiego strażnika. Walka ze strażnikiem rani gracza, a użycie Bruisewort Tonic usuwa ranę:

Pressed too hard, Iren snaps her map case shut and signals the oathbound guard.
Threats: Oathbound Guard (hostile)

You beat back Oathbound Guard, but the struggle leaves you wounded.
Status: wounded

You use Bruisewort Tonic. The wound stops bleeding.

Przykładowa komenda:

```
python3 -B -m questgen.play --series quests/generated/ash_bell
```

Automatyczne pełne przejście:

```
python3 -B -m questgen.play --series quests/generated/ash_bell --auto
```

8. Mocne i słabe strony

Mocne strony:

- Wygenerowane PDDL jest kontrolowane i łatwe do naprawienia.
- Każdy quest jest weryfikowany przez lokalny planner STRIPS.
- JSON i PDDL są kompilowane z tych samych kroków questu.
- Player używa tej samej semantyki akcji co planner.
- Player posiada opcjonalne wybory eksploracyjne i konsekwencje decyzji zamiast pojedynczej wymuszonej akcji w większości stanów.
- `gemma3:4b` jest wystarczająco mały, by działać lokalnie, i był wystarczająco szybki dla tego zadania.
- Zestaw 163 testów jednostkowych pokrywa parser, planner, domenę, generator, player, pliki wyjściowe i pipeline integracyjny (84% pokrycia kodu).

Słabe strony:

- Kompilator symboliczny ogranicza swobodę narracyjną w porównaniu z pełną, dowolną generacją LLM.
- Predykaty stage nadal prowadzą krytyczną ścieżkę questu przez kanoniczną kolejność narracyjną.
- Transfer stanu między questami jest uproszczony; ważne niesione przedmioty są wnioskowane per quest.
- Opcjonalne konsekwencje wpływają na lokalny stan, ale obecnie nie rozgałęziają końcowego celu questu.
- Tryb `full` LLM pozostaje eksperymentalny i może produkować niepoprawne schematy.

- **gemma3:4b** w trybie **compact** zmienia jedynie narracyjną “skórkę”; nie generuje strukturalnie różnych questów. Podstawowe lokacje, przedmioty, NPC i kolejność kroków zawsze wracają do deterministycznego szablonu.

9. Uwagi o wyborze modelu

Użycie angielskich promptów poprawiło niezawodność i uczyniło **gemma3:4b** wystarczającym. Większy model **qwen3.6:latest** był technicznie zainstalowany, ale nie okazał się praktyczny tutaj, ponieważ kończył się timeoutem nawet przy kompaktowych promptach. Ostatecznie wygenerowana seria jest zatem rzeczywistym uruchomieniem wspomaganym przez Ollamę z użyciem mniejszego pobranego modelu, a nie wyłącznie wynikiem fallback.

10. Podział prac

Osoba	Główny obszar	Szczegółowy opis wykonanych zadań
Paulina Hładki	Domena PDDL i solver STRIPS	Zaprojektowanie wspólnej domeny <code>mythic_quest</code> (typy, predykaty, akcje STRIPS) w pliku <code>questgen/domain.py</code> . Implementacja parsera PDDL i forward planner (BFS) w pliku <code>questgen/pddl.py</code> — parsowanie, grounding akcji, solver, weryfikacja planów. Stworzenie testów jednostkowych dla parsera i solvera.
Marta Jędrzejczak	Generator fabuł i integracja LLM	Implementacja generatora w pliku <code>questgen/generator.py</code> — integracja z API Ollamy, kompilacja <i>dramatic seed</i> do PDDL/JSON, mechanizm naprawy <code>repair_quest_schema</code> , weryfikacja <code>compile_and_verify</code> . Wygenerowanie pierwszej działającej serii <code>ash_bell</code> w trybie compact z modelem gemma3:4b .

Osoba	Główny obszar	Szczegółowy opis wykonanych zadań
Szymon Szymankiewicz	Interfejs tekstowy (player)	Implementacja silnika gry w pliku <code>questgen/play.py</code> — ładowanie JSON/PDDL, generowanie menu z grounded STRIPS actions, system narracji, dialogów, konsekwencji (rany, wrogowie, opcjonalne przedmioty). Dodanie opcji <code>--auto</code> do automatycznego przechodzenia questów. Testy dla helperów playera.
Dominik Maćkowiak	Testy, weryfikacja i raport	Stworzenie testów jednostkowych w katalogu <code>tests/</code> (w tym <code>test_output_files.py</code> i <code>test_integration.py</code>). Weryfikacja wszystkich wygenerowanych serii questów. Napisanie i skład raportu (<code>raport.md</code> → PDF).